

 [Kgreen17](#) / [AIProject](#) Public[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Security](#) [Insights](#)[AIProject](#) / [ML](#) / [K-Means Clustering.py](#)   **Kevin Green and Kevin Green** [k means](#)

adad7fc · 1 minute ago



192 lines (158 loc) · 6.86 KB

Code

Blame

Raw



```
1  import pandas as pd
2  import numpy as np
3  import matplotlib.pyplot as plt
4  import seaborn as sns
5
6  from sklearn.preprocessing import StandardScaler
7  from sklearn.cluster import KMeans
8  from sklearn.metrics import silhouette_score, silhouette_samples
9  from sklearn.decomposition import PCA
10 from sklearn.metrics import calinski_harabasz_score, davies_bouldin_score
11
12 from scipy.spatial.distance import cdist
13 from numpy.random import uniform
14
15
16 class IrisUnsupervisedClustering:
17     """
18     End-to-end Unsupervised Learning Pipeline for Iris Dataset
19     """
20
21     def __init__(self, file_path):
22         self.file_path = file_path
23         self.df = None
24         self.X = None
25         self.X_scaled = None
26         self.scaler = StandardScaler()
27         self.kmeans = None
28         self.pca = None
29         self.cluster_labels = None
30
31         # -----
32         # 1. Data Loading & Initial Exploration
33         # -----
```

```
34 ✓ def load_and_explore_data(self):
35     self.df = pd.read_csv(self.file_path)
36     print("\n--- Dataset Info ---")
37     print(self.df.info())
38     print("\n--- Basic Statistics ---")
39     print(self.df.describe())
40
41     # Visual distributions
42     self.df.hist(figsize=(10, 6))
43     plt.suptitle("Feature Distributions")
44     plt.show()
45
46     sns.pairplot(self.df.select_dtypes(include=np.number))
47     plt.show()
48
49     # -----
50     # 2. Data Cleaning & Preprocessing
51     # -----
52 ✓ def preprocess_data(self):
53     self.df.dropna(inplace=True)
54     self.X = self.df.select_dtypes(include=np.number)
55     self.X_scaled = self.scaler.fit_transform(self.X)
56     print("\nData standardized successfully.")
57
58     # -----
59     # 3. Hopkins Statistic (Cluster Tendency)
60     # -----
61 ✓ def hopkins_statistic(self, sample_size=0.5):
62     X = self.X_scaled
63     n = X.shape[0]
64     m = int(sample_size * n)
65
66     random_indices = np.random.choice(n, m, replace=False)
67     X_sample = X[random_indices]
68
69     random_points = uniform(np.min(X, axis=0), np.max(X, axis=0), (m, X.shape[1]))
70
71     u_dist = np.min(cdist(random_points, X), axis=1)
72     w_dist = np.min(cdist(X_sample, X), axis=1)
73
74     hopkins_value = np.sum(u_dist) / (np.sum(u_dist) + np.sum(w_dist))
75     print(f"\nHopkins Statistic: {hopkins_value:.4f}")
76
77     if hopkins_value > 0.7:
78         print("✓ Data is highly clusterable")
79     else:
80         print("✗ Weak clustering tendency")
81
82     return hopkins_value
```

```

83
84 # -----
85 # 4. Determine Optimal Number of Clusters
86 # -----
87 ✓ def find_optimal_k(self, max_k=10):
88     wcss = []
89     silhouette_scores = []
90
91     for k in range(2, max_k + 1):
92         km = KMeans(n_clusters=k, random_state=42)
93         labels = km.fit_predict(self.X_scaled)
94         wcss.append(km.inertia_)
95         silhouette_scores.append(silhouette_score(self.X_scaled, labels))
96
97     # Elbow Plot
98     plt.plot(range(2, max_k + 1), wcss, marker='o')
99     plt.xlabel("Number of Clusters (k)")
100    plt.ylabel("WCSS")
101    plt.title("Elbow Method")
102    plt.show()
103
104    # Silhouette Plot
105    plt.plot(range(2, max_k + 1), silhouette_scores, marker='o')
106    plt.xlabel("Number of Clusters (k)")
107    plt.ylabel("Silhouette Score")
108    plt.title("Silhouette Analysis")
109    plt.show()
110
111 # -----
112 # 5. Apply K-Means Clustering
113 # -----
114 ✓ def apply_kmeans(self, k):
115     self.kmeans = KMeans(n_clusters=k, random_state=42)
116     self.cluster_labels = self.kmeans.fit_predict(self.X_scaled)
117     self.df['Cluster'] = self.cluster_labels
118
119     print("\n--- KMeans Model Summary ---")
120     print(f"Clusters: {k}")
121     print(f"Inertia: {self.kmeans.inertia_}")
122     print(f"Iterations: {self.kmeans.n_iter_}")
123
124 # -----
125 # 6. PCA for Visualization
126 # -----
127 ✓ def apply_pca(self):
128     self.pca = PCA(n_components=2)
129     pca_data = self.pca.fit_transform(self.X_scaled)
130
131     pca_df = pd.DataFrame(pca_data, columns=['PC1', 'PC2'])

```

```

132     pca_df['Cluster'] = self.cluster_labels
133
134     sns.scatterplot(data=pca_df, x='PC1', y='PC2', hue='Cluster', palette='Set2')
135     plt.title("PCA Cluster Visualization")
136     plt.show()
137
138     # -----
139     # 7. Interpret Cluster Centers
140     # -----
141     def show_cluster_centers(self):
142         centers_scaled = self.kmeans.cluster_centers_
143         centers_original = self.scaler.inverse_transform(centers_scaled)
144
145         centers_df = pd.DataFrame(centers_original, columns=self.X.columns)
146         print("\n--- Cluster Centers (Original Scale) ---")
147         print(centers_df)
148
149     # -----
150     # 8. Cluster Profiling
151     # -----
152     def cluster_profiling(self):
153         profile = self.df.groupby('Cluster').mean()
154         print("\n--- Cluster Profiling ---")
155         print(profile)
156
157         profile.plot(kind='bar', figsize=(10, 6))
158         plt.title("Cluster Feature Comparison")
159         plt.show()
160
161     # -----
162     # 9. Silhouette Plot
163     # -----
164     def silhouette_plot(self):
165         silhouette_vals = silhouette_samples(self.X_scaled, self.cluster_labels)
166
167         y_lower = 10
168         for i in np.unique(self.cluster_labels):
169             ith_cluster_vals = silhouette_vals[self.cluster_labels == i]
170             ith_cluster_vals.sort()
171             size_cluster = ith_cluster_vals.shape[0]
172
173             y_upper = y_lower + size_cluster
174             plt.fill_betweenx(np.arange(y_lower, y_upper),
175                             0, ith_cluster_vals)
176             y_lower = y_upper + 10
177
178         plt.xlabel("Silhouette Coefficient")
179         plt.ylabel("Cluster")
180         plt.title("Silhouette Plot")
181         plt.show()

```

```
181         plt.show()
182
183     # -----
184     # 10. Conclusion Helper
185     # -----
186     def conclusion(self):
187         print("""
188             ✓ Successfully clustered Iris dataset using K-Means.
189             ✓ PCA helped visualize clear cluster separation.
190             ✓ Clusters align well with flower measurements.
191             ✓ Future work: Try DBSCAN or Hierarchical Clustering.
192             """)
```